CSP-203: Software Tools & Technologies Laboratory



Lab Session - 7

Bash Scripting: Array, Functions, Commands and Expressions

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- Arrays
- Functions
- How to write bash command
- Expression evaluation
- Regular Expressions
- Shell Commands
- Introduction to file handling in Bash

What is array in bash scripting?

- A **Bash array** is a variable that can hold **multiple values** indexed by integers or strings.
- We can store any datatype in the array.

What are the types of Bash Array?

- Indexed arrays
- Associative arrays

Need of an array in script?

- ✔ Arrays simplify list management .
- ✔ Batch operations.
- ✔ Structured data handling in scripts.

- **Declaring indexed arrays:**
  - Inline declaration:
    - my_array=(element1 element2 element3…..)
    - my_array=()
  - Explicit declaration:
    - declare -a my_array
- **Assigning elements to the array:**
  - my_array[0]=100
  - my_array[1]=Cricket
  - my_array+=(element)

```
my_array=(55 22 34 41 20)
```

```
declare -a my_array
my_array[0]=banana
my_array[1]=3
my_array[2]=swimming
my_array[3]=cricket
```

- **Accessing Indexed Array:**
  - **Single Element Access:**
    - ✔ echo “${my_array[0]}
  - **Accessing All Elements:**
    - ✔ echo “${my_array[@]}”
  - **Counting total number of elements:**
    - ✔ echo “${#my_array[@]}”

```
declare -a my_array
my_array[0]=banana
my_array[1]=3
my_array[2]=swimming
my_array[3]=cricket


echo "Third element in the array: ${my_array[2]}"
echo "All elements in the array: ${my_array[@]}"
echo "Total number of elements in array: ${#my_array[@]}"
```

```
Third element in the array: swimming
All elements in the array: banana 3 swimming cricket
Total number of elements in array: 4
```

- **Looping through indexed arrays:**
  - ${my_array[@]} Expands each element separately.
  - We can access each element using for loop.
  - ${!my_array[@]} can be used to loop over indices.

```
my_array=(55 22 34 41 20)

for item in ${my_array[@]}
do
        echo "Item: $item"
done
```

```
arindam@ARINDAM:~$ ./demo.sh
Item: 55
Item: 22
Item: 34
Item: 41
Item: 20
```

```
my_array=(55 22 34 41 20)

for i in ${!my_array[@]}
do
        echo "Item[$i]: ${my_array[$i]}"
done
```

```
arindam@ARINDAM:~$ ./demo.sh
Item[0]: 55
Item[1]: 22
Item[2]: 34
Item[3]: 41
Item[4]: 20
```

Slicing and subarrays:

- Extracting a subarray from an array.
- sub_array=(${my_array[@]:start_index:number of elements})

```
my_array=(55 22 34 41 20)

sub_array=(${my_array[@]:2:3})

echo "${sub_array[@]}"
```

```
arindam@ARINDAM:~$ ./demo.sh
34 41 20
```

- Another way to make sub_array is to make it from offset to end.

```
my_array=(55 22 34 41 20)

sub_array=(${my_array[@]:3})

echo "${sub_array[@]}"
```

```
arindam@ARINDAM:~$ ./demo.sh
41 20
```

- **Deleting elements from array:**
  - unset my_array[1] : It deletes the first indexed element from the array.
  - unset my_array : It deletes the whole array.
  - After unset, indices remain “holes” until reassigned.

```
my_array=(55 22 34 41 20)

unset my_array[1]

echo "${my_array[@]}"

unset my_array

echo "${my_array[@]}"
```

```
arindam@ARINDAM:~$ ./demo.sh
55 34 41 20
```

- **Declaring associative arrays:**
  - declare –A dict
- **Assigning elements to the array:**
  - dict[key]=element
    - Here keys are indices, it can be integer, strings etc.
- **Accessing the array:**
  - Same as the indexed array.
- **Looping over associative array:**

```
declare -A dict
dict[fruit]=banana
dict[game]=cricket
dict[color]=red

for key in ${!dict[@]}
do
        echo "$key => ${dict[$key]}"
done
```

```
arindam@ARINDAM:~$ ./demo.sh
color => red
fruit => banana
game => cricket
```

```
declare -A dict
dict[fruit]=banana
dict[game]=cricket
dict[color]=red

echo "${dict[@]}"
echo "${!dict[@]}"
echo "${#dict[@]}"
echo "${dict[fruit]}"
echo "${dict[color]}"
```

```
arindam@ARINDAM:~$ ./demo.sh
red banana cricket
color fruit game
3
banana
red
```

Functions Definition:

- Syntax 1:

  func_name(){

  Statement

  }

- Syntax 2:

  function func_name(){

  Statement

  }

- After declaring you have to call the function from the script by:

  func_name



❑ Passing arguments to a function:

- At the time of calling the function give the variable values beside the function_name.

  Syntax:

  - func_name $variable1 $variable2…

- While declaring a variable, use local, otherwise the declared variable will override the global variable.
- Static scoping, dynamic scoping is not required in this course. So, we will not look into into it.
- Use $1 to access first argument that has been passed. $2 is used to access 2nd argument and so on.

```
incr(){
        local a=$1
        ((a++))
        echo "Inside incr: $a"
}

main(){
        local a=2
        echo "Inside main: $a"
        incr $a
        echo "Inside main: $a"
}

main
```

```
arindam@ARINDAM:~$ ./demo.sh
Inside main: 2
Inside incr: 3
Inside main: 2
arindam@ARINDAM:~$
```

- **Returning value from a function:**
  - Method 1:
    - We can return the value from the function using: return $variable
    - Then we can access the value immediately after calling the function using $?
    - If we access other variable before accessing return value then, $? will print 0 after that.

```
incr(){
        local a=$1
        ((a++))
        return $a
}
main(){
        local a=2
        incr $a
        echo "Inside main local variable a: $a"
        echo "Inside after function invoked after accessing a: $?"
}
main
```

```
arindam-chakraborty@arindamchakraborty-ASUS-TUF-Gamin
ab6$ ./script1.sh
Inside main local variable a: 2
Inside after function invoked after accessing a: 0
arindam-chakraborty@arindamchakraborty-ASUS-TUF-Gamin
```

```
incr(){
        local a=$1
        ((a++))
        return $a
}
main(){
        local a=2
        incr $a
        echo "Inside main acessing returned value immediately: $?"
        echo "Inside main acessing a: $a"
}
main
```

```
arindam-chakraborty@arindamchakraborty-ASUS-TUF-Gaming
ab6$ ./script1.sh
Inside main acessing returned value immediately: 3
Inside main acessing a: 2
arindam-chakraborty@arindamchakraborty-ASUS-TUF-Gaming
```

- **Returning value from a function:**
  - Method 2:
    - We can print the value that we want to return from the function.
    - We can capture the output via command substitution.
      - val1=$(func_name arg1 arg2 …)
    - Then we can print the captured value.

```
incr(){
        local a=$1
        ((a++))
        echo "$a"
}

main(){
        local a=2
        captured_value=$(incr $a)
        echo "Inside main: $a"
        echo "Inside main(Captured Value): $captured_value"
}

main
```

```
arindam@ARINDAM:~$ ./demo.sh
Inside main: 2
Inside main(Captured Value): 3
arindam@ARINDAM:~$
```

- **Path Testing:**
  - If path is a string indicating a path, we can test if it is a valid path, the type of file it represents and the type of permissions associated with it:
    - -e path : tests if path exists
    - -f path : tests if path is a file
    - -d path : tests if path is a directory
    - -r path : tests if you have permission to read the file
    - -w path : tests if you have write permission
    - -x path : tests if you have execute permission
    - -s path : tests if the size of the file is greater than 0

- **Path Testing:**

```
if [ -f $1 ]
then
  Perform some actions
else
  echo "This script needs a file as its input dummy!"
fi
```

- **Combining Expressions:**
  - If path is a string indicating a path, we can test if it is a valid path, the type of file it represents and the type of permissions associated with it:
    - You can combine tests:

```
if [ testexp1 -a testexp2 ]
then
    cmd
fi
```

    - -a : and
    - -o : or
    - ! testexp1 : not

❑ Regular Expression Basic:

Operator	Meaning	Example	Matches
.	Any single character	a.b	aab, acb
^	Start of string	^abc	abc, but not xabc
$	End of string	abc$	abc, but not abcc
*	Zero or more of the previous character	ab*c	ac, abc, abbc
+	One or more of the previous character	ab+c	abc, abbc, but not ac
?	Zero or one of the previous character	ab?c	ac, abc
[]	Character class (one of)	[abc]	a, b, or c
[^]	Negated character class	[^abc]	Any character except a, b, or c

❑ Regular Expression Grouping:

Operator	Meaning	Example	Matches
()	Grouping	(ab)+	ab, abab, ababab
|	Or	dog|cat	dog, cat
{n}	Exactly n repetitions	a{3}	aaa
{n,}	n or more repetitions	a{2,}	aa, aaa, etc.
{n,m}	Between n and m repetitions	a{2,4}	aa, aaa, aaaa

❑ Regular Expression Escape Sequences:

Expression	Meaning	Example
\.	A literal dot (not any character)	example\.com matches example.com
\+	Escaped plus (only needed in **basic** regex)	a\+b matches a+b
\\	A literal backslash	\\ matches \

❑ Regular Expression Character Class Shorthands:

Class	Matches	Example
[[:alnum:]]	Letters and digits (A-Za-z0-9)	^[[:alnum:]]+$ matches abc123
[[:alpha:]]	Letters only (A-Za-z)	^[[:alpha:]]+$ matches abcXYZ
[[:digit:]]	Digits only (0-9)	^[[:digit:]]+$ matches 123456
[[:lower:]]	Lowercase letters (a-z)	^[[:lower:]]+$ matches abcxyz
[[:upper:]]	Uppercase letters (A-Z)	^[[:upper:]]+$ matches ABCXYZ
[[:space:]]	Whitespace (space, tab, etc.)	^[[:space:]]+$ matches tabs and spaces
[[:punct:]]	Punctuation characters	^[[:punct:]]+$ matches !@#.$
[[:xdigit:]]	Hexadecimal digits (0-9A-Fa-f)	^[[:xdigit:]]+$ matches 1A3f

❑ Regular Expression:

Expressions	Explanation
.	Matches any single character.
?	The preceding item is optional and will be matched, at most, once.
*	The preceding item will be matched zero or more times.
+	The preceding item will be matched one or more times.
{N}	The preceding item is matched exactly N times.
{N,}	The preceding item is matched N or more times.
^	Matches the empty string at the beginning of a line.
$	Matches the empty string at the end of a line.
[a-d]	Matches any one character in the range a-d.

❑ Regular Expression Example:

- First declare your expression in a variable with single quote. (Don't use double quote)
- =~ is used to compare b/w string and regular expression.
- Don't compare the expression directly without storing it in a variable

```
echo "Enter the email id: "
read n
regrex='^[a-zA-Z][a-zA-Z0-9._]+@(iitbhilai\.ac\.in|[a-zA-Z0-9]+\.[a-zA-Z]{2,})$'
if [[ $n =~ $regrex ]]
then
        echo "Valid email!"
else
        echo "Not valid email!"
fi
```

❑ Shell commands through Bash:

```
#! /bin/bash

echo "Listing all files: "
ls -l
~
```

```
arindam@ARINDAM:~$ ./demo.sh
Listing all files:
total 16
-rw-r--r-- 1 arindam arindam   0 May 15 15:25 Arindam
-rwxr-xr-x 1 arindam arindam  47 May 16 17:08 demo.sh
-rwxr-xr-x 1 arindam arindam 120 May 14 00:56 demo1.sh
-rwxr-xr-x 1 arindam arindam 211 May 16 13:05 email_check.sh
-rwxr-xr-x 1 root    root     35 May 13 17:43 iit.sh
arindam@ARINDAM:~$
```

- **How to check whether no input was given:**

```
if [ -z $FILE ]
then
  echo “No filename was given as input!”
  exit 1
fi
```

- **How to check whether the file exists or not:**

```
if [ ! -f $FILE ]
then
  echo “Error: $FILE does not exist!”
  exit 1
fi
```

- **Counting lines in a file through bash script:**

  lines=$(wc -l < $FILE)

- **Counting words in a file through bash script:**

  words=$(wc –w < $FILE)

- **Counting characters in a file through bash script:**

  chars=$(wc –m < $FILE)

- **Counting size of the file through bash script:**

  file_size=$(wc –c <$FILE)

- **Checking whether a word exists in the file or not:**

  word_count=$(grep -o “\b$word\b” $FILE | wc –l)

  - \b ensures that not substring is taken into consideration, only the given word is considered.
  - if we want to make this case sensitive then we can use grep –o –i “\b$FILE\b” $FILE | wc -l

```
word_counts=$(grep -o "\b$2\b" $1 | wc -l)

if [ $word_counts -gt 0 ]
then
        echo "$2 exists in $1!"
else
        echo "$2 does not exist in $1!"
fi
```

```
arindam@ARINDAM:~$ cat text_file.txt
I am studying at IIT Bhilai.
I am a student of computer science and engineering.
I am studying B.Tech.
I am going to home.
I am returning from home.
Our campus is beautiful.
The weather is rainy.
My roll number is 12345.
arindam@ARINDAM:~$ ./demo.sh text_file.txt am
am exists in text_file.txt!
arindam@ARINDAM:~$ ./demo.sh text_file.txt only
only does not exist in text_file.txt!
arindam@ARINDAM:~$
```

- *** in bash script:**
  - * (asterisk) is a pattern that matches every filename and folders [non-hidden] in the current directory.
  - Example: *.txt matches all file names with .txt extension at the end.
  - Example: *foo* matches all file and folder names which contains “foo” as substring.
  - Example: iit* matches all file and folder names which has “iit” as a preceding substring.

- **Print all file and directories in the current directory:**

```
arindam@ARINDAM:~$ ./demo.sh
Arindam
demo.sh
demo1.sh
directory1
email_check.sh
hello
hello.c
iit.sh
text_file.txt
arindam@ARINDAM:~$
```

```
#! /bin/bash

for f in *
do
        if [ $f ]
        then
                echo "$f"
        fi
done
```


THANK
YOU